

Algorithmics	Student information	Date	Number of session
	UO: 300737	10/02/2025	1
	Surname: Hoyos	 Escuela de Ingeniería Informática Universidad de Oviedo	
	Name: Ignacio		



Activity 1: Measuring execution times

¿How many years can we continue using currentTimeMillis()?

$2^{64}/2 = 9\,223\,372\,036\,854\,775\,807$ Max number achievable (half because of negative numbers)

$1000 * 60 * 60 * 24 * 365,25 = 31557600000$ milliseconds in a year

$9\,223\,372\,036\,854\,775\,807 / 31557600000 = 292\,271\,023$ years

Dividing the max positive possible long number between the milliseconds that a year has, we will get the amount of years that currentTimeMillis() will work. So we just have to subtract the years that it has been working to know the years left:

$292\,271\,023 - (2025-1970) = \mathbf{292270968}$ years

VECTOR 2 - Why does the measured time sometimes come out as 0?

Because the elapsed time is so low that the approximation tends to 0.

VECTOR 2 - From what size of problem (n) do we start to get reliable times?

With $n \geq 13\,000\,000$ we get times above 50 seconds, but obviously as we increase n, we increase their reliability.

Activity 2: Taking small execution times

What happens with time if the problem size is multiplied by 2? What happens with time if the problem size is multiplied by a value k other than 2? (try it, for example, for k=3 and k=4 and check the times obtained)

Firstly, it seems that the time grows slower than the size, but as n starts to have a considerable size it looks like the time accelerates and catches up to have a linear complexity relation with the size.

Algorithmics	Student information	Date	Number of session
	UO: 300737	10/02/2025	1
	Surname: Hoyos		
	Name: Ignacio		

PC => CPU: 12th Gen Intel(R) Core(TM) i5-12400 2.50 GHz, RAM: 16 GB

TABLE1 (times in milliseconds WITHOUT OPTIMIZATION)

n	Tsum	Tmaximun
10000	0,108	0,14
20000	0,210	0,286
40000	0,395	0,526
80000	0,729	1,124
160000	1,502	2,006
320000	3,033	4,099
640000	6,106	8,368
1280000	13,556	16,455
2560000	28,319	35,815
5120000	50,7	59,6
10240000	103,1	155,1
20480000	199,5	236
40960000	400	510
81920000	1 209	1 023

TABLE2 (times in milliseconds WITHOUT OPTIMIZATION)

n	TMatches1	TMatches2
10000	806	0,129
20000	3284	0,254
40000	13 855	0,527
80000	57 999	0,988
160000	OoT	2,118
320000	OoT	3,9

Algorithmics	Student information	Date	Number of session
	UO: 300737	10/02/2025	1
	Surname: Hoyos		
	Name: Ignacio		

640000	OoT	7,6
1280000	OoT	15,2
2560000	OoT	30,3
5120000	OoT	70,8
10240000	OoT	130,1
20480000	OoT	341,7
40960000	OoT	566
81920000	OoT	1 443

Conclusion

In one hand, the sum and maximum functions grows very similar, which is not a surprise because they have the same complexity($O(n)$). In the other hand, $TMatches1(O(n))$ grows incredible faster than $TMatches2(O(n^2))$, which was expected because it has a higher complexity but not this much.